

Performance Testing

Date	20 July 2026
Project Name	OptiCrop: Smart Agricultural Production Optimization Engine

Step 1: Testing Overview

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single valid prediction request	1	1	Returns a valid crop name in < 2s
2	Missing field in request payload	1	1	Returns 400 error with clear message
3	Non-numeric input value	1	1	Returns 400 error, no crash
4	Repeated sequential requests (basic load check)	20	30	All requests succeed, no server errors

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass/Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	[fill after testing]		
2	Response Time (Max)	< 5 seconds	[fill after testing]		
3	Model Accuracy (Logistic Regression)	> 90%	[fill after running <code>train_model.py</code>]		
4	Model Accuracy (Random Forest)	> 90%	[fill after running <code>train_model.py</code>]		
5	Error Rate	< 1%	[fill after testing]		

Step 4: Observations & Analysis

Both Logistic Regression and Random Forest are lightweight classical ML models, so inference time is expected to be negligible (well under 1 second) compared to network/API overhead. `train_model.py` prints a full comparison table (accuracy, precision, recall, F1) for both candidates and automatically saves the higher-accuracy model to `model.pkl` — run it once to fill in the actual numbers above. The main performance consideration for the web app itself is Flask’s development server under concurrent load — the included `Procfile` (`gunicorn app:app`) is ready for production-style deployment.

Step 5: Screenshots / Evidence

(Attach Postman collection results or terminal timing logs here.)